SYSTEM DESIGN

Design YouTube

A global video streaming platform handling 2B+ users, 500 hours/minute uploads, 1B hours/day watched

~ 45 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation & Metrics
04 · High-Level Architecture
05 · Video Upload & Ingestion
06 · Transcoding Deep Dive
07 · Adaptive Bitrate Streaming
08 · CDN Architecture
09 · Database Design
10 · Video Search
11 · Recommendation Engine
12 · Comments & Engagement
13 · Caching Strategy
14 · Failure Modes & Resilience
15 · Trade-Offs & Design Decisions
16 · Interview Playbook

01 Problem Statement

Ask 7 clarifying questions

You are asked to design YouTube, a global video streaming platform where users can upload, watch, and share videos. The interviewer expects you to start with clarifying questions to scope the problem.

Key Clarifying Questions

Question	Answer / Scope
1. Scale: how many users?	2 billion total users; ~500 million monthly active users
2. Video upload volume?	500 hours of video uploaded per minute globally
3. Video storage duration?	Indefinite (at least 10 years); some pruning of very old unwatched content
4. Geographic reach?	Global; must support playback in all regions with local CDN
5. Real-time features?	Live streaming out of scope for MVP; focus on stored video on-demand
6. Video resolution?	Support multiple: from 144p (mobile) to 4K; user device adaptivity
7. Core product focus?	Watch experience > upload experience. Search, recommendations, comments secondary.

Design Constraints (from clarifications)

- **Global scale:** 500M MAU spread across 6 continents → need geographically distributed CDN
- **Upload velocity:** 500 h/min = 30,000 h/day = 1.25M hours/day → massive ingestion pipeline
- **Watch volume:** ~1 billion video hours watched per day globally (estimated)
- **Bias towards playback:** Read-heavy system (watch:upload ratio ≈ 2000:1)
- **Quality expectations:** Viewers expect <2-second startup time, zero buffering, instant pause/resume

© Study Guide — For personal use only

02 Functional & Non-Functional Requirements

Functional Requirements

- Upload video: chunk-based resumable upload with progress tracking
- Watch video: stream with ABR (adaptive bitrate), low latency, seek support
- List videos: search by title/tags, browse by category, trending videos
- Recommend videos: personalized recommendations based on watch history & collaborative filtering
- Comments: post, reply, like comments; nested threads with pagination
- Subscriptions: subscribe to channels, get notifications for new uploads

Non-Functional Requirements

Metric	Target
Video startup latency	<2 seconds (p99)
Rebuffer rate	<0.5% of streams
Availability	99.99% (4 nines)
Search latency	<100ms (p99)
Recommendation latency	<500ms (p99)
Storage durability	99.999999999% (11 nines, triple replication)
Video encoding latency	<4 hours for 1 TB (≈ 2 Mbps processing)

© Study Guide — For personal use only

03 Capacity Estimation

Calculate QPS, storage, bandwidth

Daily Upload Volume

```
500 hours/minute × 60 minutes = 30,000 hours/day

Assuming avg video is 10 minutes:
30,000 hours = 30,000 × 60 min = 1,800,000 videos/day

Upload QPS (assuming spread over 24h = 86,400 seconds):
1,800,000 / 86,400 ≈ 21 video_uploads/second
But peak-hour ≈ 3× average = ~60-70 uploads/sec during peak
```

Daily Storage Requirement

```
Raw video storage:
30,000 hours/day × 1 GB/hour = 30 TB raw video/day

Transcoded (10 formats: 4K+1080p+720p+480p+360p+240p+144p,
each in H.264/VP9):
30 TB × ~10 transcoded formats ≈ 300 TB/day

Annual storage needed:
300 TB/day × 365 = 109.5 PB/year
~5-year retention = 547 PB (assuming linear growth)

With 3× replication for durability:
547 PB × 3 = ~1.64 EB (Exabyte)
```

Streaming (Watch) Volume & Bandwidth

```
Estimated daily watch hours: ~1 billion hours/day

Concurrent streams at peak:
1B hours/day ÷ 86,400 sec = 11.6 million concurrent viewers

Bandwidth requirement (at 2 Mbps average adaptive bitrate):
11.6M streams × 2 Mbps = 23.2 Tbps (Terabits/sec) peak

CDN needs to handle ≈2.3 Pbps global bandwidth
Accounting for PoP peering, this is ≈ 500-1000 Tbps to origin.
```


© Study Guide — For personal use only

```
Cache efficiency: ~95% hit rate on CDN edges → only 5% origin pulls
```

Database Metrics

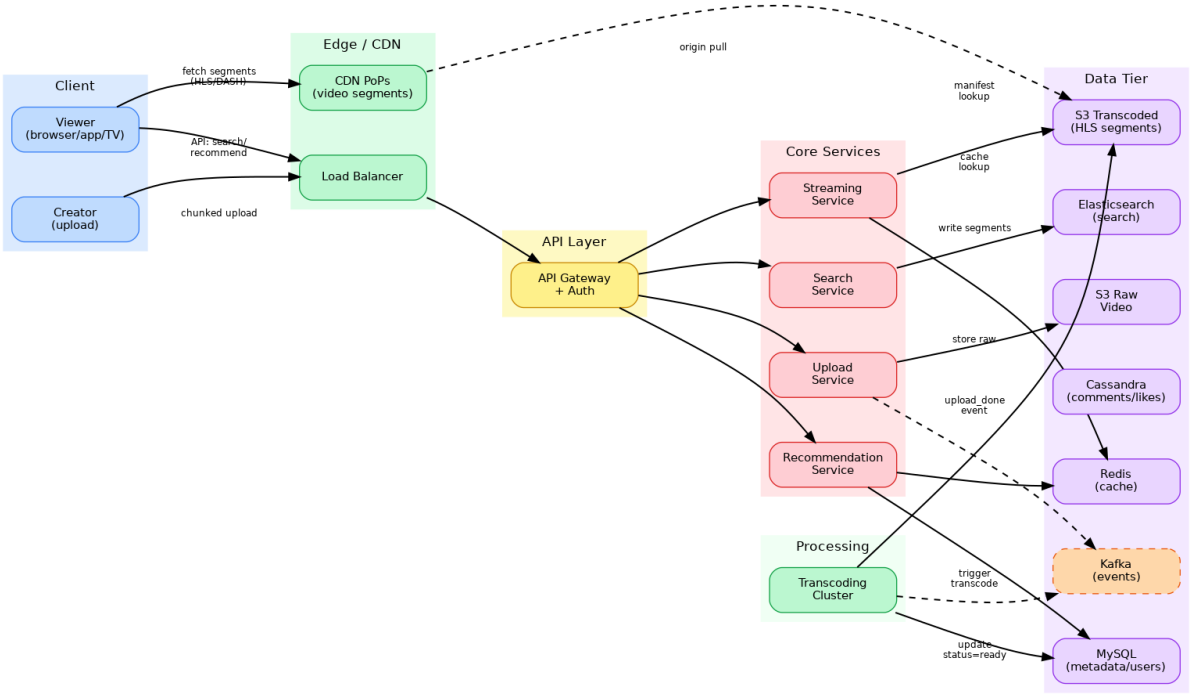
- Videos table: ~7 billion total videos (30K/day × 10 years); ~500 bytes/row = 3.5 TB
- Users table: ~2 billion users; ~1 KB/row = 2 TB
- Comments: ~50 billion comments (assume 7:1 comment:video ratio); Cassandra sharded
- Likes: ~500 billion likes across videos & comments; Redis counters + async flush

© Study Guide — For personal use only

04 High-Level Architecture

Multi-tier service design

The system is divided into several layers: client, edge/CDN, API, core services, processing, and data tier.



Component Overview

- **Client:** Web browser, mobile app, smart TV; initiates playback requests or uploads
- **Load Balancer:** Distributes inbound requests across API gateways
- **API Gateway:** Authentication, rate limiting, request routing
- **Upload Service:** Handles chunked/resumable uploads (TUS protocol)
- **Streaming Service:** Serves HLS/DASH manifests and segments
- **Search Service:** Elasticsearch for title/tag search with ranking
- **Recommendation Service:** Two-tower model (candidate generation + ranking)
- **Transcoding Cluster:** Parallel workers encoding videos into multiple formats
- **Message Queue (Kafka):** Event-driven pipeline: upload_done → transcode_trigger
- **S3:** Raw video storage (ingest) and transcoded segments (streaming)
- **CDN PoPs:** Geographically distributed Points-of-Presence caching video segments
- **MySQL:** Transactional metadata (videos, users, metadata)
- **Cassandra:** Time-series comments, scales horizontally
- **Redis:** Caching, session store, real-time counters (likes, views)
- **Elasticsearch:** Full-text search index on video metadata

 © Study Guide — For personal use only

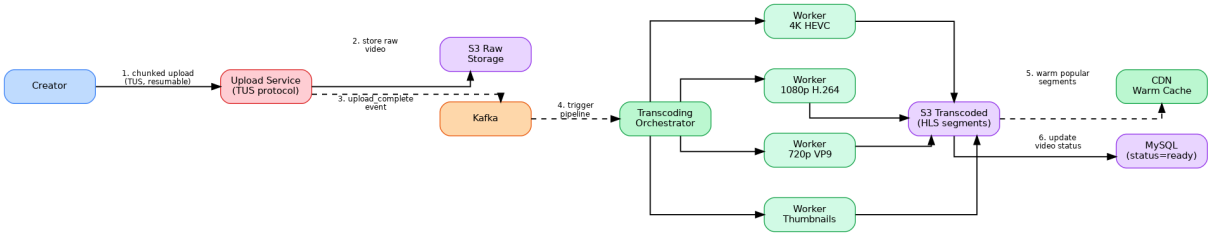
N INSIGHT

The architecture separates **write path** (upload → transcode) from **read path** (stream from CDN). This allows independent scaling of each.

© Study Guide — For personal use only

05 Video Upload & Ingestion

Resumable uploads, deduplication



TUS Protocol (Resumable Upload)

The Tus protocol (https://tus.io/) standardizes resumable uploads. YouTube-scale requires:

- **Chunking:** Split video into 5-10 MB chunks (standard chunk size)
- **Resumability:** Track which chunks uploaded; client can resume from last chunk on network failure
- **Server acknowledgment:** Upload Service returns offset and next chunk index
- **Atomic commits:** Only mark upload complete when all chunks received and checksummed

Upload Flow (Step-by-Step)

1. **Creator initiates upload:** POST /upload → receives upload_id, chunk size hints
2. **Client chunks and sends:** PATCH /upload/{id}/chunk/{n} with chunk + checksum
3. **Server acknowledges:** Returns offset progress; client buffers retries on transient error
4. **Upload completes:** Final chunk triggers verification; checksum mismatch → re-upload chunk
5. **Kafka event published:** upload_complete {video_id, creator_id, original_size} → event log
6. **Transcoding orchestrator:** Consumes event, dispatches tasks to transcoding workers
7. **Raw video stored:** S3 at s3://raw-videos/{creator_id}/{video_id}.mp4

Deduplication & Content-Addressed Storage

To avoid duplicate storage of identical videos (copies, reuploads):

- **Perceptual hashing:** Compute PHASH of uploaded video during transcoding
- **PHASH lookup:** Check if PHASH exists in Cassandra; if match found → link video_id to existing content
- **Space savings:** Avoid 50+ TB/day duplicate storage (estimated 10-15% duplicate rate)
- **Copyright enforcement:** Identical PHASHes flag potential copyright violations for review team

Handling Upload Failures

- **Transient network failure:** Client retries chunk with exponential backoff
- **Timeout on chunk:** Upload Service holds state; client resumes after 5min

 © Study Guide — For personal use only

- **Corrupted chunk (checksum mismatch):** Client re-sends; server re-computes checksum
- **Upload abandoned:** Garbage collector cleans up incomplete uploads after 7 days

> ■ **WARNING**
>
> YouTube accepts uploads from anyone. Abuse mitigation: initial uploads placed in **moderation queue** for bot detection (content-hash lookup, NSFW classifier). Cleared uploads go to transcoding.

© Study Guide — For personal use only

06 Transcoding

Codec selection, resolution ladder, cost

Codec Comparison

Codec	Bitrate (1080p)	Compression	Adoption	Notes
H.264 (AVC)	3-5 Mbps	Good (2003)	Universal (99.9%)	Highest compatibility; patent pool licenses
VP9	2-3 Mbps	Excellent	~80% devices	Open-source; better compression than H.264
AV1	1-1.5 Mbps	Excellent+	Emerging (~40%)	Next-gen; high CPU cost; 5-10× encoding time
HEVC (H.265)	2-3 Mbps	Excellent	~75% devices	Patent-heavy licensing; superior quality at bitrate

YouTube's Strategy

- Primary: H.264 (universal fallback for all browsers)
- Secondary: VP9 (Chrome, Firefox; saves ~30% bandwidth)
- Tertiary: HEVC (Apple devices; iOS/macOS, saves ~40% vs H.264)
- Experimental: AV1 (next-gen, still too slow; reserved for pre-transcoding)

Resolution Ladder

Resolution	Bitrate (H.264)	Use Case
4K (2160p)	15-25 Mbps	High-end monitors, TVs; < 5% of views
1440p	8-12 Mbps	High-end displays; ~3% of views
1080p (Full HD)	4-6 Mbps	Desktop/laptop; ~40% of views
720p (HD)	2.5-4 Mbps	Tablet, TV; ~35% of views
480p	1-1.5 Mbps	Mobile (good connection); ~15% of views
360p	0.5-1 Mbps	Mobile (cellular); ~5% of views

© Study Guide — For personal use only

Resolution	Bitrate (H.264)	Use Case
240p	0.3-0.5 Mbps	Mobile (2G/poor); ~1% of views
144p	0.15-0.3 Mbps	Emergency fallback; <0.1% of views

Per-Title Encoding (PTE)

Not all videos need all resolutions. YouTube uses ML to predict optimal ladder:

- High-motion video (sports, action): needs more bitrate at each resolution
- Low-motion video (talking head): can use lower bitrate with same quality
- Animated content: compresses better; lower bitrate needed
- PTE algorithm: sample-based analysis → predicts optimal resolution, codec, bitrate
- Cost savings: ~20% reduction in storage & encoding time

Parallel Transcoding DAG

```
Upload → Check Format → Extract Thumbnail → {
Encode H.264 1080p,
Encode H.264 720p,
Encode H.264 360p,
Encode VP9 1080p,
Encode VP9 720p,
...
} → HLS Packaging → S3 Upload → CDN Warm
```

Uses DAG (Directed Acyclic Graph) dependency resolution. All codec/resolution pairs encode in parallel (100+ tasks per video).

Encoding Cost & Metrics

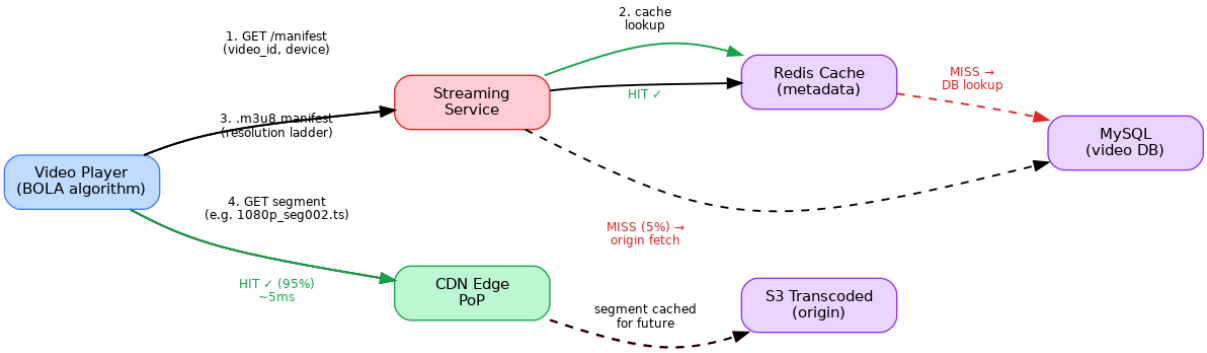
- 1 hour video → ~2-3 CPU-hours of compute (H.264 1080p, ~2 Mbps output)
- Total transcoding: 30,000 hours/day × 2.5 CPU-hours = 75,000 CPU-hours/day
- On 500 machines (8-core = 4000 CPU cores total): 75,000 / 4000 ≈ 19 hours → <1 day queue
- Cost: AWS = ~$0.10/CPU-hour; YouTube spends ~$7,500/day on transcoding alone (rough estimate)

> ✓ **TIP**
> Transcoding is the most expensive & time-consuming step in the upload pipeline. Parallel encoding & smart codec selection are critical for cost control.

 © Study Guide — For personal use only

07 Adaptive Bitrate Streaming (ABR)

HLS/DASH, BOLA algorithm



HLS vs DASH

Standard	Creator	Segment Duration	Manifest Format	Adoption
HLS	Apple (2009)	6-10 seconds	.m3u8 playlist	iOS (99%); Safari
DASH	MPEG (2011)	2-4 seconds	.mpd XML	Android (~80%); Edge, Chrome

YouTube uses both. Modern players: DASH primary, HLS fallback for Safari.

HLS Manifest Structure

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:10
#EXT-X-MEDIA-SEQUENCE:0

#EXTINF:10.0,
https://cdn.youtube.com/v/{video_id}/seg_0.ts
#EXTINF:10.0,
https://cdn.youtube.com/v/{video_id}/seg_1.ts
#EXTINF:10.0,
https://cdn.youtube.com/v/{video_id}/seg_2.ts
...
#EXT-X-ENDLIST
```

Example .m3u8 manifest for 1080p stream

Manifest Variants (Adaptive Ladder)

© Study Guide — For personal use only

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
https://cdn.youtube.com/v/{video_id}/1080p/manifest.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720
https://cdn.youtube.com/v/{video_id}/720p/manifest.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=854x480
https://cdn.youtube.com/v/{video_id}/480p/manifest.m3u8
...
```

Master manifest offering multiple quality tiers

BOLA (Buffer-Based Rate Adaptation) Algorithm

Client-side algorithm that decides which bitrate to request next:

- **Measurement:** Player monitors buffer level (seconds of video buffered)
- **Buffer target:** Maintain 10-30 seconds; above = raise bitrate, below = lower bitrate
- **Safety margin:** Never fill buffer >80% (risk of segment not finishing in time)
- **Rebuffer penalty:** If buffer hits 0, immediately drop to lowest bitrate for safety
- **Convergence:** BOLA converges to bitrate that matches available bandwidth

Typical Client Request Flow

1. Player requests master manifest: GET /api/manifest/{video_id}
2. Streaming Service checks Redis cache (hit rate ≈ 90%)
3. Return .m3u8 with variants (1080p, 720p, 480p)
4. Player selects initial bitrate (usually 720p or lowest)
5. Player fetches segment: GET CDN /v/{video_id}/720p/seg_0.ts
6. CDN PoP serves from cache (95% hit); MISS → fetch from origin S3
7. Player measures download time; BOLA adjusts bitrate for next segment
8. Rebuffer detected → BOLA immediately switches to 480p

Startup Latency Optimization

- **Manifest pre-warming:** Cache manifests for trending videos in Redis (10-min TTL)
- **CDN pre-positioning:** Warm CDN edges for top 1M videos during off-peak hours
- **Initial segment selection:** Always request lowest quality first (fast download), then upgrade
- **Player buffering:** Buffer ≥2 segments (≈20 seconds) before starting playback

© Study Guide — For personal use only

N INSIGHT

Target: <2s startup latency. Achieved via: (1) cached manifests, (2) CDN pre-positioning, (3) low-quality initial segment, (4) parallel segment downloads.

© Study Guide — For personal use only

08 CDN Architecture

Multi-tier, ISP peering, cache warming

CDN Topology (3-Tier)

- **Tier 1 (Regional):** Large hub PoPs in major metro areas (NYC, LA, London, Tokyo, Sydney, São Paulo). 50-100 Gbps edge capacity each.
- **Tier 2 (Metro):** Smaller PoPs in secondary cities. ~10-20 Gbps each.
- **Tier 3 (ISP peering):** Direct connections to ISPs for last-mile optimization. Low-cost caching boxes (~1 Gbps).

Cache Hit Strategy

- **Most-popular 1%:** Cache indefinitely on all Tier-1 PoPs (100% hit rate)
- **Popular 10%:** LRU cache on Tier-1 (≈95% hit rate)
- **Long-tail 89%:** Cache on Tier-2/ISP nodes or fetch from origin on demand
- **Expected avg hit rate:** 95% on Tier-1, 75% on Tier-2, 50% on Tier-3

Origin Pull (Cache Miss Handling)

```
1. Client requests: GET cdn.youtube.com/v/{vid}/720p/seg_42.ts
2. Edge PoP checks local cache → MISS
3. Edge asks parent PoP (Tier-1) → MISS
4. Tier-1 asks origin (S3) → FETCH
5. S3 returns segment; cached at Tier-1 & edge simultaneously
6. Client receives segment (origin latency + RTT ≈ 50-200ms)
```

Smart Cache Warming for Viral Videos

- **Detection:** Trending detector monitors recommendation service & search logs
- **Threshold:** >100K searches/hour or viral recommendation triggers warming
- **Action:** Batch job fetches all segments for viral video from S3 → caches on top 10 Tier-1 PoPs
- **Lead time:** ~30 minutes before content goes live (coordinated with creators for live events)
- **Result:** Viral videos achieve 99.9% cache hit rate immediately upon launch

■ ANALOGY

ISP Peering (Open Connect): YouTube partners directly with ISPs to place caching boxes in ISP data centers. Reduces ISP transit costs & improves user experience. ~70% of all YouTube traffic flows through Open Connect.

© Study Guide — For personal use only

09 Database Design

Sharding, choice of data stores

MySQL — Videos & Metadata

```
CREATE TABLE videos (
video_id BIGINT PRIMARY KEY,
creator_id BIGINT NOT NULL,
title VARCHAR(200),
description TEXT,
duration_sec INT,
upload_time TIMESTAMP,
status ENUM('processing', 'ready', 'failed'),
view_count BIGINT DEFAULT 0,
thumbnail_url VARCHAR(500),
PRIMARY KEY (video_id),
KEY (creator_id, upload_time)
);
```

Videos table schema

Sharding Strategy

- Shard by: video_id % 256 (256 shards total)
- Reasoning: video_id is immutable, queries always include video_id
- Replication: 3× (primary + 2 replicas); async replication to replicas
- Growth: 7B videos / 256 shards ≈ 27M videos per shard; manageable on commodity hardware

Cassandra — Comments & Engagement

```
CREATE TABLE comments (
video_id BIGINT,
parent_comment_id BIGINT,
comment_id BIGINT,
author_id BIGINT,
text TEXT,
timestamp TIMESTAMP,
PRIMARY KEY ((video_id), timestamp DESC, comment_id)
) WITH CLUSTERING ORDER BY (timestamp DESC);
```


© Study Guide — For personal use only

Rationale for Cassandra

- Write-heavy: Comments are posted continuously (millions/day)
- Time-series: Natural clustering by timestamp (most recent first)
- Horizontal scaling: Partition by video_id; as more videos added, add Cassandra nodes
- Durability: Built-in replication (RF=3); no need for separate backup
- Query pattern: 'Get top 100 comments for video_id' — perfect for Cassandra

Redis — Counters & Caching

```
# Real-time counters:
SET video:{video_id}:views 12345
SET video:{video_id}:likes 999
SET comment:{comment_id}:likes 42

# Metadata caching (1-hour TTL):
SET video:{video_id}:meta '{...}' EX 3600

# Session store (1-day TTL):
SET session:{token} '{user_id}:{timestamp}' EX 86400
```

Async Flush to DB

- Like counts incremented in Redis (fast, in-memory)
- Background job flushes increments to Cassandra every 5 minutes
- Rationale: Exact like counts not critical; batch writes reduce DB load
- Trade-off: Eventual consistency (5-min lag) vs. operational simplicity

> **N INSIGHT**
> Database design trade-off: Use MySQL for structured, relational data (videos, users); Cassandra for high-write, time-series data (comments); Redis for real-time counters & caching.

© Study Guide — For personal use only

10 Video Search

Elasticsearch ranking, autocomplete

Elasticsearch Inverted Index

```
PUT /videos_index
{
'mappings': {
'properties': {
'title': {'type': 'text', 'analyzer': 'standard'},
'tags': {'type': 'keyword'},
'transcript': {'type': 'text'},
'view_count': {'type': 'long'},
'engagement': {'type': 'float'}
}
}
}
```

Elasticsearch mapping for video search

Ranking Algorithm

- **TF-IDF:** Term frequency-inverse document frequency scores matches
- **Boost factors:** Title matches > tag matches > transcript matches
- **Engagement signal:** Multiplier based on view_count, like_rate, watch_time (recent videos boosted)
- **Personalization:** Re-rank by user's watch history (implicit collab filtering)

Autocomplete Typeahead

As user types, suggest trending searches ("trending" = high search volume in last hour):

```
# Redis Trie for autocomplete:
ZADD trending:q 1000 'machine learning'
ZADD trending:q 950 'machine learning tutorial'
ZADD trending:q 850 'machine learning models'

# Query: 'machine l...'
ZRANGEBYLEX trending:q '[machine l' '[machine m' LIMIT 0 10
# Returns top 10 trending queries matching prefix
```


© Study Guide — For personal use only

Search Pipeline (End-to-End)

1. User types query in search box → debounced (100ms)
2. GET /search/autocomplete?q=machine%20l → Redis trie lookup
3. Return top 10 trending searches matching prefix (< 10ms)
4. User submits search → GET /search?q=machine%20learning&page;=1
5. API hits Elasticsearch; returns 20 matching videos, scored
6. Backend hydrates metadata (view_count, thumbnail) from Redis cache
7. Frontend renders with title, channel, view count, upload date

> **✓ TIP**
> Search at YouTube scale: ~3 billion searches/day. Cached trending searches reduce Elasticsearch load by ~80%. Autocomplete latency must be <50ms (perceived lag).

© Study Guide — For personal use only

11 Recommendation Engine

Two-tower model, candidate generation, ranking

Two-Tower Architecture

YouTube recommendation engine uses a two-stage approach:

- **Stage 1 (Candidate Generation):** Retrieve ~10K candidate videos from user's preferences (fast, recall-focused)
- **Stage 2 (Ranking):** Score those 10K via complex model; return top 20 (accuracy-focused)

Candidate Generation

- **Method:** Collaborative filtering (user-based or item-based)
- **Similarity metric:** Cosine similarity on watch history embeddings
- **Execution:** Approximate nearest neighbor (ANN) search via Faiss or Elasticsearch
- **Latency:** ~50-100ms to retrieve 10K candidates
- **Refresh:** Nightly job recomputes embeddings for all users

Ranking (LightGBM Model)

```
# Example features (150 total):
- user_watch_history_avg_rating
- user_subscription_overlap (% channels user follows)
- video_view_count_normalized
- video_engagement_rate (likes/views)
- video_upload_recency (log days since upload)
- user_video_category_affinity
- temporal_trend (video virality slope)
- user_device_type
- user_country
- user_language_preference
```

LightGBM ranking features

Model trained on historical click-through data (CTR). Target: maximize watch time.

Exploration vs. Exploitation

- **Exploitation (80%):** Recommend videos matching user's known preferences
- **Exploration (20%):** Recommend novel categories to serendipitously engage user
- **Bandit algorithm:** Thompson Sampling or UCB to balance tradeoff

© Study Guide — For personal use only

- **Goal:** Avoid filter bubbles; expose users to diverse content

> **N INSIGHT**
> Recommendation model is the biggest lever for engagement. Tiny improvements in CTR/watch-time multiply to billion-hour gains. A/B testing constant.

© Study Guide — For personal use only

12 Comments & Engagement

Nested threads, spam filtering

Comment Sorting & Threading

- **Top-level comments:** Sorted by engagement score (likes – dislikes + age decay)
- **Nested replies:** Default sort by chronological (newest); option to sort by engagement
- **Display limit:** Show top 20 comments; paginated for more
- **Threading depth:** Limit to 3 levels (comment → reply → reply-to-reply) to prevent deep trees

Like Counters (Eventual Consistency)

```
# User clicks like:
1. Increment Redis counter: INCR comment:{id}:likes
2. Return new count to user immediately (optimistic)
3. Log event: comment_liked {user_id, comment_id, timestamp}
4. Background job (every 5 min): Flush Redis → Cassandra

# Edge case: Reload page
- Redis value may be stale (5-min max)
- Acceptable for social feature (users expect eventual consistency)
```

Spam & Abuse Filtering

- **Content filters:** Regex for profanity, hateful content; real-time classifier
- **Author filters:** Block known bad-actor accounts; shadow-ban for suspicious activity
- **ML detector:** BERT-based toxic comment classifier (accuracy ~95%)
- **Manual review queue:** Disputed comments routed to human reviewers
- **User reporting:** 3+ reports auto-hides comment; escalates to review queue

■ NOTE

Comments are second-order engagement feature. Most users don't read; optimized for trending & viral comments. Trade-off: over-moderation suppresses expression; under-moderation risks toxicity.

© Study Guide — For personal use only

13 Caching Strategy

Multi-tier: Redis, CDN, browser

Cache Hierarchy

Layer	TTL	Typical Hit Rate	Backend
Browser cache	1 day	~80%	LocalStorage / IndexedDB
Redis (application)	1 hour	~85%	In-memory, distributed
CDN edge	7 days	~95%	PoP storage (segments)
Origin (S3)	—	100%	Source of truth

Redis Cache Patterns

```
# Metadata (1-hour TTL):
key: video:{video_id}:meta
value: {title, creator, duration, ...} (JSON)

# Manifest (1-hour TTL):
key: manifest:{video_id}
value: .m3u8 content (text)

# Trending videos (5-min TTL):
key: trending:videos
value: [list of video_ids] (zset sorted by score)

# User session (24-hour TTL):
key: session:{token}
value: {user_id, permissions, ...}
```

Cache Invalidation Strategy

- **TTL-based (lazy):** Metadata expires after 1 hour (acceptable for most use cases)
- **Event-based (eager):** Creator edits title → publishes event → workers delete cache key
- **Versioned keys:** video:{video_id}:meta:v{version} to handle rapid updates
- **Cache stampede:** Use probabilistic expiration or distributed locks to avoid thundering herd

CDN Cache Warming

- **Off-peak hours:** Batch job fetches popular video segments from S3 → CDN edges
- **Trending detector:** Monitors search logs; marks videos trending >100K searches/hour

© Study Guide — For personal use only

- **Proactive warming:** When video reaches 50% of peak, pre-warm all segments to top 50 CDN PoPs
- **Cost trade-off:** Warm storage ≈ $0.02/GB/month; avoid 1000s of origin pulls ≈ $1/Gbps

© Study Guide — For personal use only

14 Failure Modes & Resilience

8 key failure scenarios

Common Failure Scenarios

Failure	Impact	Detection (MTTR)	Recovery
Transcoding worker crash	1-2 videos stall; 30-min queue buildup	5 min (job timeout)	Retry on different worker; preserve state in Kafka
CDN PoP outage (1 region)	Users rerouted to next-nearest PoP; +50ms latency for 10M users	10 sec (health checks)	Geo-failover via DNS; auto-reprovision PoP in 1 hour
MySQL shard leader crash	Writes blocked to 1 shard; reads from replicas work	30 sec (heartbeat)	Promote replica to leader; re-sync from standby
Elasticsearch cluster split	Search unavailable; queries timeout	2 min (cluster discovery)	Partition detection; redirect to backup cluster
Redis cache evicted (OOM)	Cache hit rate drops 90% → 30%; manifest requests spike to origin	Immediate (metrics)	Add Redis node; replication sync; drain 40% data
Origin S3 rate-limited	Segment fetch slow; player stalls after 30s buffer	20 sec (client latency increase)	Activate surge-pricing quota; cache more on CDN edges
Kafka broker down (1 of 3)	No message loss (replication=3); throughput reduced 33%	15 sec (ZK detection)	Rebalance; reassign partitions; bring up new broker
BGP leak / ISP routing issue	Global availability drop ~1-5%; affects users in specific regions	5 min (latency monitoring)	Failover to alternate AS; coordinate with transit provider

■ WARNING

Design principle: Expect failures. Multi-region redundancy, graceful degradation, & automated failover are non-negotiable. Aim for 99.99% availability (52 minutes downtime/year).

© Study Guide — For personal use only

15 Trade-Offs & Design Decisions

Comparing alternatives

Choice: MySQL vs. NoSQL for Videos Table

Aspect	MySQL (Relational)	NoSQL (DynamoDB/Cassandra)
Schema flexibility	Fixed schema; strict typing	Flexible; dynamic attributes
Query flexibility	JOIN queries; complex aggregations	Limited to key-lookup; no JOINs
Consistency	ACID; strong consistency	Eventually consistent
Scalability	Sharding required; complex	Horizontal scaling (auto-sharding)
Operational complexity	High (PITR, replication, backups)	Lower (managed service)

YouTube Decision: MySQL for videos table (structured, ACID guarantees for uploads). Cassandra for comments (high-write, no relational queries needed).

Choice: Custom CDN vs. Third-Party (CloudFlare, Akamai)

- **Custom CDN (YouTube choice):** Full control over cache logic, codec selection, ISP peering. Cost ≈ $2-5 per TB. Capex-heavy; 2-3 year payoff at scale.
- **Third-party CDN:** Opex model; no ops burden. Cost ≈ $20 per TB. Suitable for <100 Tbps load.
- **Breakeven:** ~500-1000 Pbps → in-house CDN cheaper than third-party.
- **YouTube:** 2.3 Pbps streaming bandwidth → custom CDN economical.

Choice: HLS vs. DASH as Primary Format

- **DASH:** Better compression, shorter segment duration, more efficient ABR. Industry standard.
- **HLS:** Apple-friendly; all iPhones/iPads support natively.
- **YouTube decision:** DASH as primary; HLS fallback for 1% Safari traffic. Reduces encoding jobs by 10%.

Choice: Manifest Caching TTL (1 hour vs. 5 minutes vs. no cache)

- **No cache:** Each manifest request hits origin → high load; <100ms latency.
- **5-min TTL:** Stale manifests possible (video metadata updated but not reflected for 5 min); balance consistency & load.
- **1-hour TTL:** Higher hit rate; significant staleness risk for fast-changing metadata.

© Study Guide — For personal use only

• **YouTube choice:** 1-hour TTL + event-based invalidation. 90% of manifests don't change; trade latency for load reduction.

 © Study Guide — For personal use only

16 Interview Playbook

45-minute execution plan

Key Metrics to Memorize

Metric	Value	Context
Daily video uploads	500 hours/minute = 30K hours/day	Input velocity
Concurrent viewers (peak)	11.6 million	Read QPS
Video upload QPS	21 avg, 60-70 peak	Write QPS
Daily watch hours	~1 billion	Total engagement
Raw storage per day	30 TB	Ingestion capacity
Transcoded storage per day	300 TB	After encoding
5-year total storage	547 PB (3× replicated = 1.64 EB)	Long-term capex
Streaming bandwidth (peak)	2.3 Pbps	CDN capacity
CDN cache hit rate	~95%	Efficiency

45-Minute Timeline

Time	Topic	Depth
0-3 min	Problem statement & clarifying questions (7 questions)	1-2 min per question
3-8 min	Functional & non-functional requirements	Quick bulleted list
8-15 min	Capacity estimation (QPS, storage, bandwidth)	Walk through math; write on whiteboard
15-22 min	High-level architecture (draw diagram)	4 key layers: client, API, services, data
22-28 min	Deep dive into one area (transcoding OR streaming)	Codec choice, workers, DAG, CDN cache hit
28-35 min	Database design (MySQL vs. Cassandra vs. Redis)	Sharding strategy, CAP theorem
35-42 min	Failure modes & trade-offs (1-2 scenarios)	Graceful degradation, eventual consistency

© Study Guide — For personal use only

Time	Topic	Depth
42-45 min	Address interviewer questions	Listen for hints; adapt deeper topics

Interviewer Red Flags

- **'How would you scale to 10B users?'** → Interviewer testing horizontal scaling. Emphasize sharding, replication, geo-distribution.
- **'What if CDN goes down?'** → Resilience. Prepare multi-region failover, graceful degradation (lower quality), origin fallback.
- **'How do you handle a viral video?'** → Real-time adaptation. Cache warming, auto-scaling, surge pricing, traffic shifting.
- **'What's the most expensive component?'** → Transcoding (~$7.5K/day) or CDN bandwidth (~$1M+/day). Show cost awareness.

Closing Statement

Summarize in 30 seconds: **'YouTube is fundamentally a video storage and delivery system. The core tradeoff is between (1) encoding cost (transcoding), (2) storage cost (S3), and (3) bandwidth cost (CDN). We optimize each via parallelization, intelligent caching, and codec selection. Reliability comes from multi-region redundancy and graceful degradation. If I had more time, I'd deep-dive into recommendation ML, real-time metrics, or live streaming.'**

© Study Guide — For personal use only